ROTEIRO DO PROJETO

DE DESENVOLVIMENTO DE SOLUÇÃO COMPUTACIONAL

Modelo de documento para elaboração e entrega do projeto

Identificação	Preenchimento
Nome do projeto	PORTA A PORTA
Integrantes	LUIZ GUILHERME DE ALMEIDA FERNANDES BRUNO RIBEIRO GARCIA LUÍS EDUARDO BRITO COSTA MELO
Turma	GPE02M80044
Professor	RANYELSON NERES CARVALHO
Data de entrega	15/08/2026

Este documento deve refletir o que foi efetivamente planejado, construído, integrado e testado pelo grupo.

Orientações gerais

- Utilize este roteiro como estrutura mínima obrigatória do documento do projeto.
- Substitua os textos entre colchetes pelas informações reais do projeto.
- Mantenha coerência entre requisitos, diagramas, banco de dados, código-fonte, testes e funcionalidades demonstradas.
- Inclua imagens, diagramas, links e evidências sempre que solicitado.
- Não apresente como concluída uma funcionalidade que não esteja implementada e demonstrável no MVP.
- Caso algum item não se aplique ao projeto, registre a justificativa no próprio tópico.

Importante: documentação, protótipos, telas isoladas, scripts ou trechos de código não caracterizam, por si só, um MVP funcional. Os componentes principais da solução devem estar integrados e permitir a execução do fluxo central do sistema.

1. REQUISITOS

1.1 Visão e escopo

1.1.1 Declaração do problema

O desenvolvimento da solução computacional foi motivado pela desorganização e ineficiência no comércio informal praticado dentro de condomínios residenciais. Atualmente, a comercialização de produtos e serviços é realizada de forma adaptada através de grupos de mensagens instantâneas, como o WhatsApp. Esse formato atual gera séria poluição nos canais de comunicação do condomínio levando à perda de avisos importantes e sobrecarregando a gestão do síndico, além de provocar erros em pedidos e falta de clareza nos preços pela ausência de um catálogo estruturado. Essa rotina impacta negativamente tanto os moradores vendedores quanto os compradores. Uma solução computacional contribui diretamente para minimizar esse cenário ao centralizar as ofertas em uma vitrine digital segura e exclusiva, garantindo a automação do fluxo de pedidos vinculados à unidade do morador e despoluindo os canais oficiais de comunicação.

1.1.2 Visão da solução

Objetivo geral da solução:

Prover uma vitrine digital segura e exclusiva para a comunidade do condomínio, organizando o comércio interno e automatizando a gestão de pedidos entre moradores vizinhos.

Público-alvo:

Moradores de condomínios residenciais (compradores e vendedores/empreendedores) e a administração do condomínio (síndicos e gestão).

Principais benefícios esperados:

- Centralização e Organização das Ofertas: Disponibilização de uma vitrine digital padronizada e exclusiva com catálogo fixo, fotos, preços e carrinho de compras.
- Automação e Precisão na Gestão de Pedidos: Acompanhamento eficiente do fluxo e status das solicitações, eliminando erros no pedido e vinculando cada compra diretamente à unidade/bloco do morador.
- Despoluição dos Canais Oficiais de Comunicação: Redução drástica do envio de anúncios em grupos de WhatsApp do condomínio, evitando a perda de mensagens importantes da comunidade e reduzindo a sobrecarga da administração/síndico.

1.1.3 Escopo

Necessidade identificada	Funcionalidade proposta
Ter um catálogo fixo com fotos e preços	Vitrine Digital
Realizar e vincular pedidos à unidade do morador	Carrinho de Compras e Finalização por Unidade
Notificar novos pedidos e acompanhar entregas	Gerenciador de Status do Pedido
Garantir acesso restrito e seguro aos moradores	Autenticação de Usuários e Isolamento por Condomínio

Itens incluídos no escopo

- Criação de Vitrine Digital: Cadastro de produtos por moradores empreendedores com fotos, descrição e preços fixos.
- Gestão de Carrinho de Compras: Seleção de itens e fechamento de pedidos vinculados obrigatoriamente à identificação da unidade (bloco/apartamento) do morador comprador.

- Gerenciador de Status do Pedido: Painel para vendedores e compradores acompanharem o andamento, aceite e entrega dos pedidos.
- Notificações internas no aplicativo: Avisos exibidos dentro do próprio app sobre a chegada de novos pedidos e atualizações de status, consultados quando o usuário abre ou atualiza o aplicativo.

Itens fora do escopo

- Processamento de Pagamentos Online: Não inclui integração com gateways de pagamento (cartão de crédito, Pix automático, etc.). Toda a transação financeira ocorre diretamente entre os moradores (modo offline).
- Logística de Entrega Externa: O sistema não realiza gestão de frete, entregadores externos ou rastreamento de transporte fora do condomínio.
- Notificações Push: O sistema não envia alertas via push (Expo Push Notifications/FCM), pois esse recurso exige build nativo do aplicativo e integração com serviço externo. Os avisos ficam restritos às notificações internas do aplicativo.

1.2 Especificação de requisitos

1.2.1 Histórias de usuário

ID	História de usuário	Prioridade
HU01	Como Morador Comprador, eu quero consultar a vitrine digital do condomínio filtrada por categorias , para visualizar de forma rápida os produtos e serviços ativos oferecidos exclusivamente pelos meus vizinhos	Alta
HU02	Como Morador Comprador, eu quero realizar um pedido selecionando uma janela de horário para entrega ou retirada , para agendar o recebimento da mercadoria de forma organizada e compatível com a minha rotina	Alta
HU03	Como Morador Vendedor, eu quero cadastrar e manter anúncios de produtos respeitando o limite máximo de 15 itens ativos , para divulgar meu catálogo comercial de forma limpa e organizada dentro da comunidade do condomínio	Alta
HU04	Como Morador Vendedor, eu quero gerenciar o painel de entregas e atualizar o status do pedido em tempo real, gerando notificações internas no aplicativo, para otimizar minha fila de produção e manter o comprador informado sobre o progresso da entrega	Média

1.2.2 Regras de negócio

ID	Regra de negócio
RN01	Limite de Anúncios Ativos por Vendedor: O morador cadastrado como vendedor pode manter, no máximo, 15 itens ativos simultaneamente em sua vitrine digital. Caso atinja essa marca, o sistema bloqueia a inserção de novos produtos até que um anúncio antigo seja desativado, pausado ou excluído.
RN02	Moderação de Conteúdo Gerado por Usuários (Denúncias e Avaliações): Para preservar a confiança da comunidade, toda ação de moderação sobre conteúdo sensível gerado por usuários segue o princípio de que apenas o síndico tem poder de decisão final. (a) Denúncias e quarentena automática: um anúncio que receber denúncias de 3 CPFs distintos é automaticamente movido para o estado "Quarentena" (oculto da vitrine), gerando alerta para análise humana do síndico, que decide manter a remoção, restaurar o anúncio ou aplicar outra penalidade. (b) Avaliações e indelebilidade: avaliações (nota 1-5 e comentário) são definitivas e não podem ser editadas pelo autor após o envio; apenas o síndico pode removê-las, mediante solicitação do vendedor afetado ou de ofício, quando identificar conteúdo abusivo ou de má-fé. (c) Privacidade do denunciante/avaliador: a denúncia é sempre exibida sem o nome, a unidade ou qualquer outro dado que identifique o denunciante, tanto para os demais moradores e para o vendedor denunciado quanto para o síndico, que visualiza na fila de moderação apenas o anúncio, o motivo e a data da denúncia. Da mesma forma, a identidade de quem avalia não é exposta a outros moradores nem ao vendedor avaliado. O sistema mantém internamente o vínculo com o autor (CPF/ID) somente para permitir a contagem de CPFs distintos e impedir denúncias duplicadas, sem exibi-lo em nenhuma tela. As decisões de moderação do síndico (quarentena, restauração de anúncio e remoção de avaliação) são registradas em log de auditoria, com a respectiva justificativa.
RN03	Restrição de Transação Financeira e Logística Integrada: O sistema opera estritamente sob escopo informacional e de gestão interna, não realizando processamento de pagamentos online (como cartões ou Pix integrados) e não oferecendo suporte a frete ou logística externa . Todo o pagamento e entrega devem ocorrer de forma direta e offline entre os vizinhos.

1.2.3 Requisitos não funcionais

ID	Categoria	Requisito não funcional
RNF01	Usabilidade	O sistema deve manter o usuário informado sobre novos pedidos e mudanças de status por meio de notificações internas no aplicativo, sem depender de serviço externo de push.
RNF02	Segurança	O acesso deve ser rigidamente restrito à comunidade local, exigindo a vinculação obrigatória de todo pedido à identificação da unidade do morador (Ex: Bloco/Apartamento). Toda a comunicação de dados deve trafegar sob o protocolo HTTPS para garantir criptografia básica, e o banco de dados deve utilizar mecanismos de Multitenancy (isolamento lógico de dados) por meio da chave estrangeira condominio_id em todas as entidades do sistema.
RNF03	Compatibilidade	O sistema deve garantir a persistência integral de todos os dados de catálogos e pedidos diretamente no banco de dados relacional, mitigando quaisquer chances de perda de dados ou inconsistência em cenários de quedas de energia ou do servidor.
RNF04	Escalabilidade	A infraestrutura inicial do servidor em nuvem (com custo estimado de R$ 50,00/mês) deve ser dimensionada para suportar a operação e o tráfego simultâneos de até 20 condomínios de pequeno porte .
RNF05	Integridade e Desempenho	O sistema deve impor limites estritos no upload de mídia para os anúncios, recusando arquivos que não sejam do formato de imagem ou que excedam o tamanho máximo de 5MB por foto.

2. PROJETO DE SOFTWARE

2.1 Projeto de interface com usuário

Tela 1: Tela de Login

Item	Descrição
Objetivo da tela	Formulário de autenticação dos usuários.
Usuário que utilizará	Compradores e Vendedores.
Principais funcionalidades	Realiza a autenticação dos usuários no sistema.
Protótipo	https://freeimage.host/i/CZYQvxp

Tela 2: Tela de Pedidos

Item	Descrição
Objetivo da tela	Exibir ao Usuário os pedidos feitos no sistema.
Usuário que utilizará	Compradores.
Principais funcionalidades	Exibe informações e status dos pedidos feitos.
Protótipo	https://freeimage.host/i/img-pedidos.CZYtDl9

Tela 3: Tela de Perfil

Item	Descrição
Objetivo da tela	Exibir ao Usuário o perfil no aplicativo.
Usuário que utilizará	Compradores e Vendedores.
Principais funcionalidades	Exibe informações sobre um Usuário.
Protótipo	https://freeimage.host/i/CZYDtXR

Tela 4: Tela Principal

Item	Descrição
Objetivo da tela	Tela principal do Aplicativo.
Usuário que utilizará	Compradores e Vendedores.
Principais funcionalidades	Exibe informações sobre produtos da vizinhança e menus de navegação.
Protótipo	https://freeimage.host/i/CZYb5LQ

2.2 Arquitetura lógica

2.2.1 Estilo arquitetural

Item	Descrição
Arquitetura adotada	Cliente-servidor com API REST; backend organizado em módulos de rotas por recurso, que concentram a validação das entradas e as regras de negócio e acessam o banco diretamente pelo ORM, com middlewares compartilhados de autenticação, autorização por perfil e tratamento de erros.
Justificativa da escolha	O app mobile (cliente) consome uma API REST central, permitindo que múltiplos condomínios compartilhem o mesmo backend com isolamento lógico por condominio_id (RNF02). A organização em módulos de rotas com acesso direto ao ORM mantém cada regra de negócio (RN01-RN03) junto ao recurso que ela afeta e reduz a quantidade de camadas e de código para o porte do MVP, enquanto os middlewares evitam a repetição de autenticação e tratamento de erros. Como o servidor não mantém estado de sessão, é possível escalá-lo horizontalmente conforme exigido pelo RNF04 (até 20 condomínios).

2.2.2 Componentes da aplicação

Componente/Módulo	Responsabilidade
Módulo de Autenticação	Login, cadastro e vinculação do usuário à unidade e ao condomínio (RNF02). Os perfis são hierárquicos: o vendedor tem também as permissões do comprador, e o síndico as do vendedor e do comprador.
Módulo de Vitrine/Catálogo	CRUD de anúncios com uma foto por produto armazenada no servidor (RNF05), controle do limite de 15 itens ativos (RN01) e busca por categoria (HU01).
Módulo de Pedidos	Carrinho, checkout vinculado à unidade, escolha de dia e período de entrega (HU02) e status do pedido (HU04). Nenhum usuário compra o próprio produto.
Módulo de Moderação	Denúncias, quarentena automática e avaliações, conforme RN02.
Módulo de Notificações	Registro e exibição de avisos internos no aplicativo (tela de avisos com contador de não lidos): novo pedido para o vendedor, mudança de status para o comprador, quarentena automática para o síndico e o vendedor e decisão do síndico para o vendedor (RNF01).
Painel do Síndico	Fila de moderação com denúncias exibidas sem identificação do denunciante, decisão sobre quarentena/avaliações e log de auditoria das ações de moderação.

2.2.3Diagramas

Diagrama de Classes

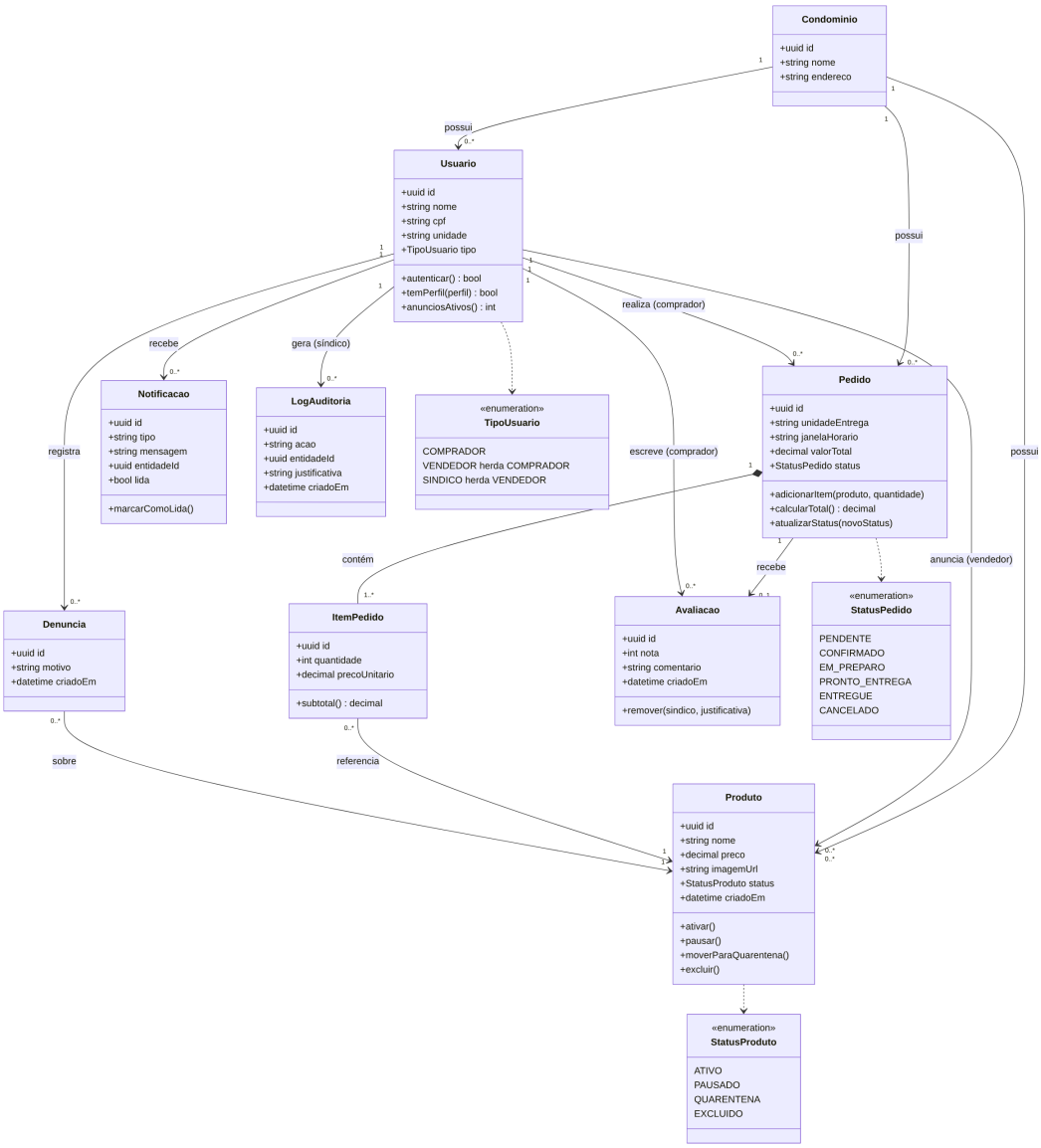
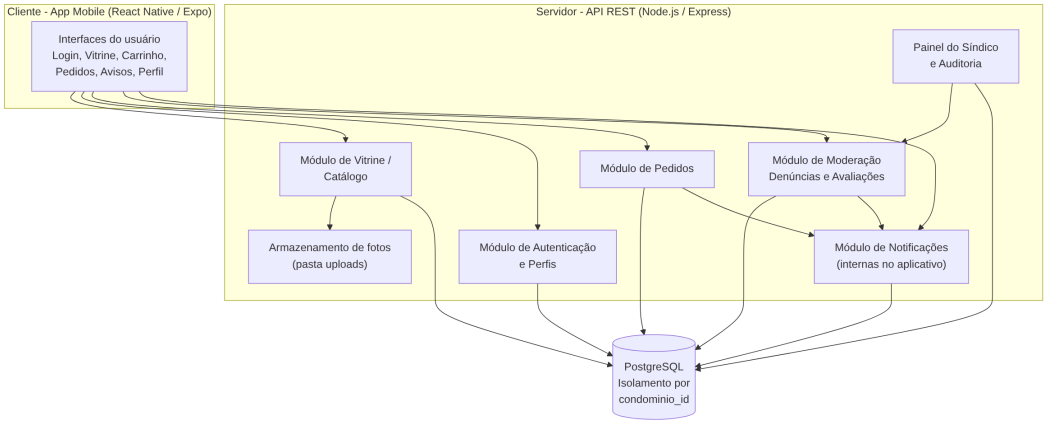


Diagrama de Pacotes ou Componentes



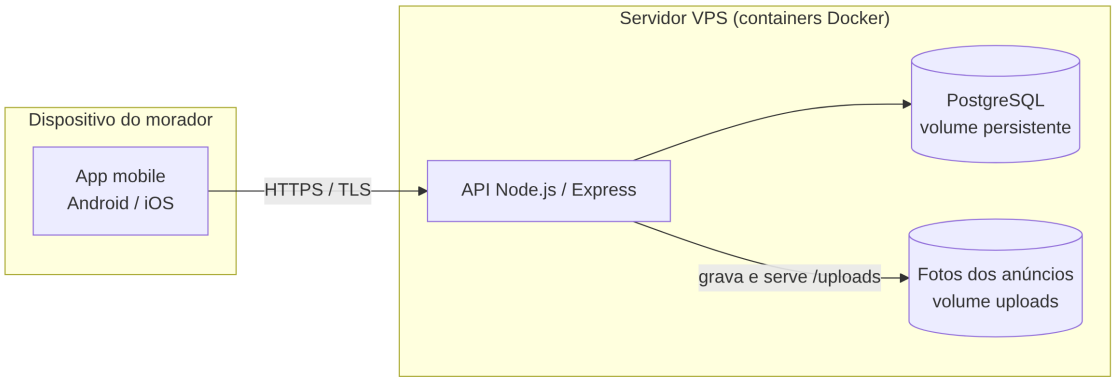
2.2.4 Tecnologias utilizadas

Categoria	Tecnologia
Linguagem de programação	JavaScript / TypeScript
Front-end	React Native (Expo)
Back-end	Node.js + Express
Banco de dados	PostgreSQL
Frameworks	Express, Prisma (ORM)
APIs/Serviços externos	Não se aplica: o sistema não depende de serviços externos. As fotos dos anúncios ficam armazenadas no próprio servidor.
Controle de versão	Git + GitHub

2.3 Arquitetura física

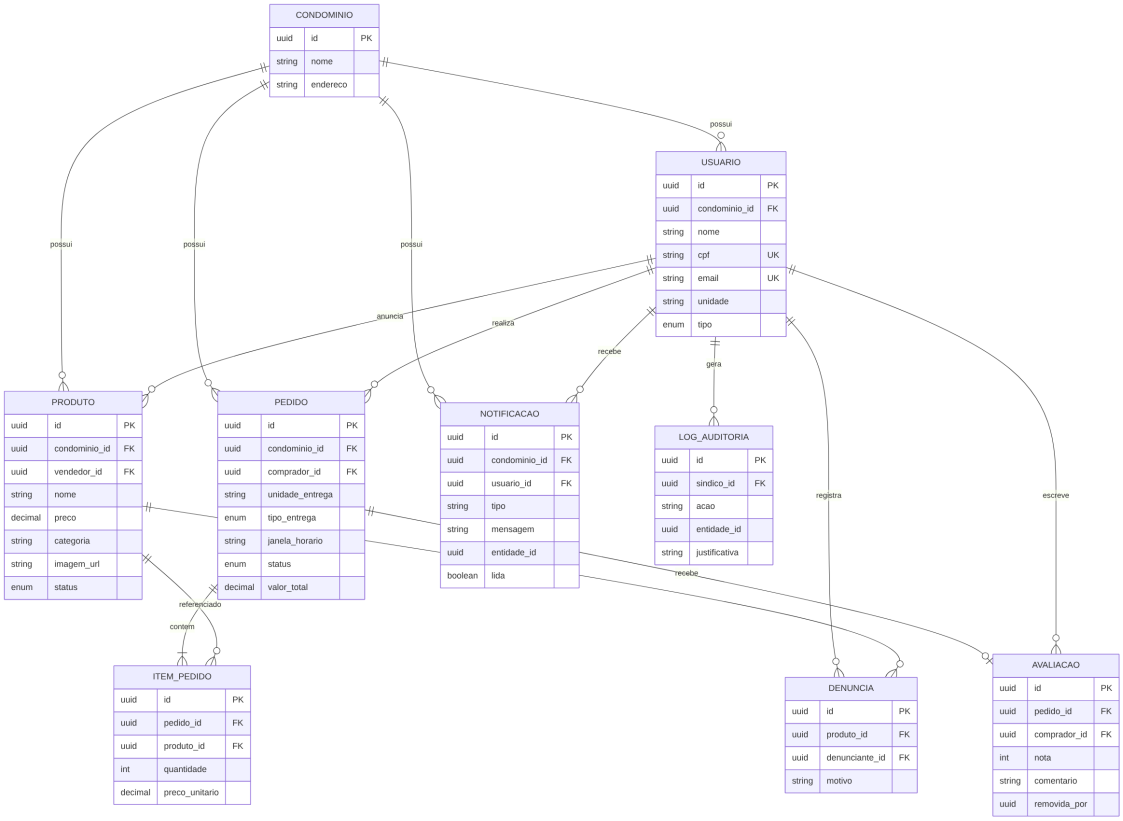
Recurso	Descrição
Cliente	App mobile React Native (Expo), instalado no smartphone do morador (Android/iOS).
Servidor	Aplicação Node.js/Express executada em container Docker em um servidor VPS.
Banco de dados	Instância PostgreSQL executada em container no mesmo servidor VPS, com volume persistente.
Rede/Internet	Comunicação via HTTPS (TLS) entre app e API, conforme RNF02.
Serviço de hospedagem	Servidor VPS em provedor de nuvem (~R$ 50/mês), dimensionado para até 20 condomínios (RNF04).
Outros recursos	Pasta de uploads no servidor, em volume persistente, com as fotos dos anúncios (uma por produto, até 5 MB - RNF05), servidas pela própria API.

Diagrama de implantação



2.4 Projeto de banco de dados

2.4.1 Modelo Entidade-Relacionamento



2.4.2 Descrição das tabelas

Tabela	Finalidade
condominio	Isola cada condomínio (multitenancy — RNF02).
usuario	Moradores e síndico, vinculados à unidade e ao condomínio. O tipo define o perfil, com herança de permissões (COMPRADOR, VENDEDOR e SINDICO).
produto	Anúncios da vitrine, com limite de 15 ativos por vendedor (RN01) e o caminho da foto armazenada no servidor.
pedido	Pedidos vinculados à unidade do comprador (HU02).
item_pedido	Itens de cada pedido (relação N:N entre pedido e produto).
avaliacao	Avaliações de vendedores, imutáveis pelo autor (RN02).
denuncia	Denúncias de anúncios, usadas para disparar a quarentena (RN02).
notificacao	Avisos internos exibidos no aplicativo, com mensagem, referência ao pedido ou anúncio e indicação de leitura (RNF01).
log_auditoria	Registro das ações de moderação do síndico (quarentena, restauração de anúncio e remoção de avaliação), com justificativa (RN02).

2.4.3 Estrutura das tabelas

Os identificadores (id e chaves estrangeiras) são do tipo UUID, gerados pela aplicação, o que evita expor a sequência de registros e facilita a combinação de dados entre condomínios.

Tabela: produto

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
condominio_id	UUID	FK -> condominio	Isolamento multitenant
vendedor_id	UUID	FK -> usuario, NOT NULL	Dono do anuncio
nome	VARCHAR	NOT NULL	Nome do produto
preco	DECIMAL	NOT NULL	Preco fixo
status	VARCHAR	NOT NULL	ativo / pausado / quarentena / excluido (RN02)
imagem_url	VARCHAR	NULL	Caminho da foto no servidor (RNF05)
criado_em	TIMESTAMP	NOT NULL	Data de cadastro

Tabela: denuncia

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
produto_id	UUID	FK -> produto, NOT NULL	Anuncio denunciado
denunciante_id	UUID	FK -> usuario, NOT NULL	Autor da denuncia (uso interno, nao exibido - RN02)
motivo	VARCHAR	NOT NULL	Categoria da denuncia
criado_em	TIMESTAMP	NOT NULL	Data da denuncia
-	-	UNIQUE(produto_id, denunciante_id)	Impede denuncia duplicada do mesmo usuario (RN02)

Tabela: avaliacao

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
pedido_id	UUID	FK -> pedido, NOT NULL	Pedido avaliado
comprador_id	UUID	FK -> usuario, NOT NULL	Autor da avaliacao
nota	INTEGER	NOT NULL, CHECK 1-5	Nota de 1 a 5
comentario	TEXT	-	Comentario livre
removida_por	UUID	FK -> usuario (nullable)	Sindico que removeu, se houver (RN02)
criado_em	TIMESTAMP	NOT NULL	Nao editavel apos insercao (RN02)

Tabela: log_auditoria

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
sindico_id	UUID	FK -> usuario, NOT NULL	Sindico que realizou a acao
acao	VARCHAR	NOT NULL	Ex: aplicou quarentena, restaurou anuncio, removeu avaliacao
entidade_id	UUID	NOT NULL	Anuncio/avaliacao moderado
justificativa	TEXT	NOT NULL	Motivo da decisao de moderacao (RN02)
criado_em	TIMESTAMP	NOT NULL	Data/hora da acao

3. CONSTRUÇÃO

3.1 Código fonte

Item	Informação
Repositório do projeto	https://github.com/luizgaf/porta-a-porta
Linguagem principal	TypeScript, no front-end e no back-end.
Frameworks e bibliotecas utilizadas	Front-end: React Native com Expo, Expo Router (navegação), Zustand (estado), AsyncStorage (persistência local), Axios (HTTP), Expo Image Picker (foto do produto) e Expo Font com as fontes Fraunces e DM Sans. Back-end: Node.js com Express, Prisma ORM, Zod (validação), JSON Web Token e bcryptjs (autenticação), Helmet, CORS e Morgan. Testes de unidade com Jest e ts-jest. Banco de dados: PostgreSQL, executado via Docker Compose.

3.1.1 Organização do código

O repositório é único e está dividido em três pastas principais: backend (API REST), frontend (aplicativo móvel) e docker (banco de dados local). O back-end é organizado em módulos de rotas por recurso, e o front-end segue o roteamento por arquivos do Expo Router, com uma tela por arquivo.

Pasta/arquivo	Conteúdo e responsabilidade
backend/src/index.ts	Inicialização do servidor Express: middlewares globais (Helmet, CORS e Morgan), verificação de saúde (/health), registro das rotas e conexão com o banco.
backend/src/routes/	Um módulo de rotas por recurso (auth, condominio, usuario, produto, pedido, denuncia, avaliacao, auditoria e notificacao), com validação das entradas, regras de negócio e acesso ao banco. O arquivo index.ts registra todos sob o prefixo /api.
backend/src/middleware/	auth.ts (autenticação por token e autorização por perfil, com a hierarquia de perfis) e errorHandler.ts (classe de erro da aplicação e tratamento centralizado de erros).
backend/src/services/	Funções compartilhadas entre as rotas: notificacoes.ts (gravação dos avisos internos) e imagens.ts (validação do formato e do tamanho da foto, gravação e remoção na pasta de uploads).
backend/prisma/	schema.prisma (modelos e enums), migrations (scripts SQL de criação e atualização do banco), seed.ts (dados iniciais) e seed-images (fotos padrão dos produtos de exemplo).
backend/uploads/	Fotos dos produtos enviadas pelos vendedores, servidas pela API no endereço /uploads (não versionada).
frontend/app/	Telas do aplicativo, uma por arquivo: (auth) para boas-vindas, login, cadastro e seleção de perfil e de condomínio; (tabs) para vitrine, busca, carrinho, pedidos, painel do vendedor, moderação e perfil; product, order, notifications e settings para cadastro e detalhe de produto, detalhe de pedido, avisos e configurações.
frontend/components/	Componentes de interface reutilizáveis (Button, Input, Card, Avatar, CategoryChip, DoorTag, ProductCard, ProductTile e ProductImage), componentes das telas de acesso e o provedor de autenticação.
frontend/hooks/	useAuth (sessão e perfil do usuário) e useApi (chamadas à API agrupadas por recurso).
frontend/store/	Estado global com Zustand, persistido no dispositivo: authStore (sessão) e cartStore (carrinho).
frontend/utils/	api.ts (cliente HTTP com Axios, que envia o token em cada requisição e trata as respostas de erro) e funções auxiliares de formatação, seleção e envio de fotos e paginação das listas.
frontend/constants/ e frontend/types/	Constantes, rótulos e tokens de design (cores, tipografia e espaçamentos); tipos TypeScript compartilhados entre as telas.
docker/	docker-compose.yml do PostgreSQL, script de inicialização do banco e run.sh, que sobe e para o container e prepara o banco (setup: migrações e dados iniciais; reset: recria o banco do zero).

Pasta/arquivo	Conteúdo e responsabilidade
adb/	Scripts tunnel.sh e tunnel.cmd, que redirecionam as portas da API (3000) e do Expo (8081) para um dispositivo Android conectado por USB, para testar o aplicativo em aparelho físico (com o comando npm run start:local).
TUTORIAL.md	Guia de execução do projeto e contas de teste.

3.1.2 Padrões de qualidade adotados

- Padronização dos nomes de variáveis: variáveis e funções em camelCase (condominioId, valorTotal); componentes, tipos e modelos em PascalCase (ProductCard, Usuario); valores de enum em maiúsculas (ATIVO, PENDENTE); tabelas e colunas do banco em snake_case (condominio_id, criado_em). Os nomes de domínio são escritos em português.
- Organização em classes, módulos ou componentes: no back-end, um módulo de rotas por recurso; no front-end, uma tela por arquivo, componentes de interface em components/ui e estado global em stores separados.
- Separação de responsabilidades: autenticação, autorização por perfil e tratamento de erros ficam em middlewares; a validação das entradas é feita por esquemas declarados no início de cada módulo de rotas; a persistência passa pelo ORM, e as funções usadas por mais de um módulo (avisos e fotos) ficam em services. No aplicativo, as telas não acessam a API diretamente, e sim por meio do hook useApi.
- Reutilização de código: funções e classes compartilhadas por todas as rotas (asyncHandler, AppError, authMiddleware e requireRole); componentes de interface, tokens de design e tipos reutilizados entre as telas.
- Tratamento de erros: tratamento centralizado na API, que devolve respostas padronizadas com mensagem, código do erro e o status HTTP adequado (400 para validação, 401 e 403 para acesso, 404 para registro inexistente e 409 para duplicidade). No aplicativo, as chamadas são feitas com try/catch e os erros das ações do usuário são exibidos em alertas.
- Comentários quando necessários: comentários curtos em português identificam cada endpoint e marcam os pontos em que uma regra de negócio é aplicada (por exemplo, a RN01 no cadastro e na ativação de produtos).
- Controle de versão: Git, com repositório público no GitHub e histórico no ramo master; as alterações são desenvolvidas em ramos de trabalho e integradas por pull request. A estrutura do banco também é versionada, por meio das migrações do Prisma. As configurações sensíveis (arquivo .env) não são versionadas; o repositório traz apenas o modelo .env.example.
- Organização de commits: mensagens descritivas, em sua maioria com prefixo de tipo (feat, fix, chore e docs), indicando a natureza de cada alteração.
- Tipagem estática: TypeScript em modo strict no front-end e no back-end, com tipos compartilhados para as entidades do sistema.

3.2 Teste de unidade

Os testes de unidade foram implementados com Jest no back-end (pasta backend/tests), com o banco de dados simulado, e executados com o comando npm test (Jest). Os testes TU19 a TU32 cobrem as funcionalidades acrescentadas depois da primeira rodada (avisos internos, remoção de avaliação pelo síndico, foto do produto e hierarquia de perfis). O TU18, referente ao carrinho no aplicativo, ainda não foi implementado.

ID	Unidade testada	Cenário	Resultado esperado	Resultado obtido
TU01	Criação de produto (POST /produtos)	Vendedor com 15 produtos ativos tenta cadastrar mais um (RN01).	Requisição recusada com status 400 e código LIMIT_EXCEEDED; nenhum produto é criado.	Sucesso
TU02	Alteração de status do produto (PATCH /produtos/:id/status)	Vendedor com 15 produtos ativos tenta reativar um produto pausado (RN01).	Status 400 e código LIMIT_EXCEEDED; o produto permanece pausado.	Sucesso
TU03	Alteração de status do produto (PATCH /produtos/:id/status)	Vendedor tenta mover o próprio produto para quarentena.	Status 403 e código FORBIDDEN; apenas o síndico altera para esse status.	Sucesso
TU04	Criação de pedido (POST /pedidos)	Comprador envia pedido com dois produtos e quantidades diferentes (HU02).	Pedido criado (201) com status PENDENTE; valor total igual à soma de preço x quantidade, calculada no servidor; preço unitário gravado em cada item.	Sucesso
TU05	Criação de pedido (POST /pedidos)	Pedido inclui um produto pausado ou de outro condomínio.	Status 400 e código PRODUTO_INVALIDO; nenhum pedido é criado.	Sucesso
TU06	Atualização de status do pedido (PATCH /pedidos/:id/status)	Vendedor avança o pedido de PENDENTE para CONFIRMADO (HU04).	Status 200; o pedido passa a CONFIRMADO.	Sucesso
TU07	Atualização de status do pedido (PATCH /pedidos/:id/status)	Vendedor tenta passar o pedido de PENDENTE direto para ENTREGUE.	Status 400 e código INVALID_TRANSITION; o status não muda.	Sucesso
TU08	Atualização de status do pedido (PATCH /pedidos/:id/status)	Comprador tenta confirmar o próprio pedido.	Status 403 e código FORBIDDEN; o comprador só pode cancelar.	Sucesso
TU09	Registro de denúncia (POST /denuncias)	Três usuários distintos denunciam o mesmo anúncio (RN02).	Após a segunda denúncia o produto continua ATIVO; após a terceira, passa a QUARENTENA.	Sucesso
TU10	Registro de denúncia (POST /denuncias)	O mesmo usuário denuncia o mesmo anúncio duas vezes.	Status 409 e código DUPLICATE_DENUNCIA; a contagem de denúncias não aumenta.	Sucesso
TU11	Listagem de denúncias (GET /denuncias)	Síndico consulta a fila de moderação (RN02).	A resposta não contém nome, unidade ou identificador do denunciante.	Sucesso
TU12	Criação de avaliação (POST /avaliacoes)	Comprador tenta avaliar um pedido que ainda não foi entregue.	Status 404 e código PEDIDO_NOT_ELIGIBLE; nenhuma avaliação é criada.	Sucesso
TU13	Edição de avaliação (PUT /avaliacoes/:pedidoId)	Autor tenta alterar a nota depois do envio (RN02).	Status 403 e código AVALIACAO_IMUTAVEL; nota e comentário permanecem inalterados.	Sucesso
TU14	Middleware de autenticação (authMiddleware)	Requisição sem token e requisição com token inválido.	Status 401 com os códigos UNAUTHORIZED e INVALID_TOKEN; a rota protegida não é executada.	Sucesso
TU15	Middleware de autorização (requireRole)	Comprador acessa rota restrita ao síndico.	Status 403 e código FORBIDDEN.	Sucesso
TU16	Tratamento de erros (errorHandler)	Erro de validação, registro duplicado e erro inesperado.	Status 400 (VALIDATION_ERROR, com a lista de campos), 409 (DUPLICATE_ENTRY) e 500 (INTERNAL_ERROR), respectivamente.	Sucesso

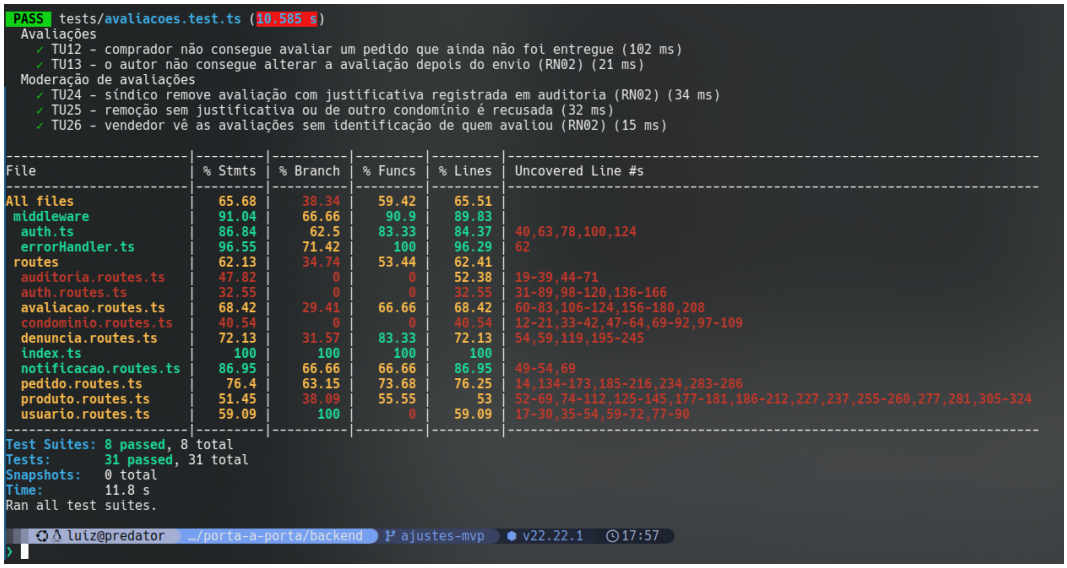
ID	Unidade testada	Cenário	Resultado esperado	Resultado obtido
TU17	Consulta de produto (GET /produtos/:id)	Usuário consulta um produto de outro condomínio (RNF02).	Status 404 e código PRODUTO_NOT_FOUND.	Sucesso
TU18	Carrinho de compras (cartStore)	O mesmo produto é adicionado duas vezes e depois tem a quantidade alterada para zero.	As quantidades são somadas e os totais recalculados; ao zerar, o item sai do carrinho.	Pendente
TU19	Notificações (POST /pedidos)	Comprador faz um pedido (HU04).	O vendedor recebe um aviso de novo pedido, com referência ao pedido.	Sucesso
TU20	Notificações (PATCH /pedidos/:id/status)	Vendedor muda o status do pedido (HU04).	O comprador recebe um aviso com o novo status.	Sucesso
TU21	Notificações (POST /denuncias)	Terceira denúncia coloca o anúncio em quarentena (RN02).	Síndico e vendedor são avisados; o aviso não identifica os denunciantes.	Sucesso
TU22	Listagem de avisos (GET /notificacoes)	Usuário consulta os próprios avisos.	Retorna só os avisos do usuário, com a contagem de não lidos.	Sucesso
TU23	Leitura de aviso (PATCH /notificacoes/:id/lida)	Usuário tenta marcar como lido o aviso de outra pessoa.	Status 404; o aviso continua não lido.	Sucesso
TU24	Remoção de avaliação (DELETE /avaliacoes/:pedidoId)	Síndico remove uma avaliação com justificativa (RN02).	Avaliação marcada como removida e log de auditoria REMOVER_AVALIACAO gravado com a justificativa.	Sucesso
TU25	Remoção de avaliação (DELETE /avaliacoes/:pedidoId)	Remoção sem justificativa ou de avaliação de outro condomínio.	Status 400 (VALIDATION_ERROR) e 404, respectivamente; nada é alterado.	Sucesso
TU26	Listagem de avaliações (GET /avaliacoes)	Vendedor consulta as avaliações recebidas (RN02).	A resposta não contém nome, unidade ou identificador de quem avaliou.	Sucesso
TU27	Foto do produto (PUT /produtos/:id/imagem)	Dono envia uma foto JPEG válida (RNF05).	Arquivo gravado na pasta de uploads e imagem_url atualizada no produto.	Sucesso
TU28	Foto do produto (PUT /produtos/:id/imagem)	Envio de arquivo que não é imagem e de foto maior que 5 MB (RNF05).	Status 415 (FORMATO_INVALIDO) e 413 (ARQUIVO_MUITO_GRANDE); nada é gravado.	Sucesso
TU29	Criação de pedido (POST /pedidos)	Vendedor compra produto de outro morador.	Pedido criado (201), pois o vendedor herda as permissões do comprador.	Sucesso
TU30	Criação de pedido (POST /pedidos)	Síndico compra produto de outro morador.	Pedido criado (201), pois o síndico herda as permissões do comprador.	Sucesso
TU31	Criação de pedido (POST /pedidos)	Usuário tenta comprar o próprio produto.	Status 400 e código PRODUTO_PROPRIO; nenhum pedido é criado.	Sucesso
TU32	Hierarquia de perfis (temPerfil)	Verificação das permissões de cada perfil.	Cada perfil tem as permissões do anterior, mas não as do seguinte.	Sucesso

Indicador	Resultado
Quantidade de testes executados	31
Testes com sucesso	31
Testes com falha	0
Taxa de sucesso	100%
Cobertura de código, quando disponível	65,51% das linhas (rotas e middlewares do back-end)

Evidências



Execução de npx jest --verbose --coverage no back-end em 14/10/2026: os 31 testes aprovados, agrupados por arquivo.



Relatório de cobertura da mesma execução: 65,51% das linhas; 8 suítes e 31 testes aprovados.

3.3 Interfaces com usuário

Esta seção não precisa apresentar novamente todas as telas implementadas, pois a interface será avaliada diretamente no software em execução.

Item	Informação
Situação da implementação	Concluída
Principais telas implementadas	Acesso: boas-vindas, login, cadastro e escolha do perfil e do condomínio. Abas: vitrine (grade de produtos com foto e filtro por categoria), busca, carrinho (com escolha do tipo de entrega, do dia e do período), pedidos (compras e vendas), painel do vendedor, moderação (denúncias, quarentena e avaliações, para o síndico) e perfil. Demais telas: detalhe e cadastro de produto (com foto), detalhe do pedido com a linha do tempo do status, avisos (notificações internas) e

	configurações (privacidade, ajuda e sobre).

Durante a apresentação, o grupo deverá demonstrar as interfaces funcionando no sistema.

3.4 Banco de dados

O banco de dados foi implementado em PostgreSQL, executado em um container Docker com volume persistente. A estrutura é definida no schema do Prisma e criada por migrações versionadas no repositório. Os identificadores são do tipo UUID e todas as entidades de negócio carregam o condominio_id, que isola os dados de cada condomínio (RNF02).

Item	Informação
SGBD utilizado	PostgreSQL 16, em container Docker, acessado pela API por meio do Prisma ORM.
Situação da implementação	Concluída
Local dos scripts no repositório	https://github.com/luizgaf/porta-a-porta, nas pastas backend/prisma (schema, migrações e dados iniciais) e docker (container do banco e scripts de execução).

Os scripts utilizados em cada etapa são os seguintes:
• Criação do banco: docker/docker-compose.yml, que cria o container, o usuário e o banco porta_a_porta, e docker/init.sql, executado na primeira inicialização para habilitar as extensões usadas.
• Criação das tabelas: backend/prisma/schema.prisma (modelos, enums, chaves e índices) e a migração backend/prisma/migrations/20260922011543_init, com o script SQL que cria as tabelas.
• Atualização da estrutura: a migração 20261014180000_notificacoes_imagem_produto, que cria a tabela de notificações, acrescenta a coluna imagem_url aos produtos e remove a coluna push_token, que não é mais usada. As migrações são aplicadas com o comando prisma migrate deploy.
• Inserção de dados: backend/prisma/seed.ts, executado com npm run prisma:seed, junto com as fotos da pasta backend/prisma/seed-images.
• Execução: docker/run.sh setup sobe o banco, aplica as migrações e insere os dados iniciais; docker/run.sh reset apaga o volume e recria o banco do zero.

Dados iniciais

Sim. O seed cria os dados necessários para executar e demonstrar a aplicação:
• 10 condomínios de teste (seed-cond-1 a seed-cond-10), cada um com o seu síndico (sindico1@pap.com a sindico10@pap.com, senha sindico123);
• no primeiro condomínio, um comprador (comprador1@pap.com, senha comprador123) e um vendedor (vendedor1@pap.com, senha vendedor123);
• 15 produtos desse vendedor, distribuídos em 8 categorias, cada um com foto.

Pedidos, denúncias, avaliações e avisos não são pré-cadastrados: eles são gerados durante a demonstração, a partir das contas de teste. O seed pode ser executado mais de uma vez sem duplicar registros, pois atualiza os dados já existentes.

---

4. TESTES

4.1 Teste funcional

Verifique se as funcionalidades implementadas atendem aos requisitos especificados. Os testes devem considerar a aplicação como caixa-preta, observando entradas, ações e resultados.

ID	Requisito/História	Cenário de teste	Resultado esperado	Resultado obtido	Situação
TF01	HU01	[Cenário]	[Resultado]	[Resultado]	Aprovado/Reprovado
TF02	HU02	[Cenário]	[Resultado]	[Resultado]	Aprovado/Reprovado
TF03	HU03	[Cenário]	[Resultado]	[Resultado]	Aprovado/Reprovado
TF04	HU04	[Cenário]	[Resultado]	[Resultado]	Aprovado/Reprovado

Indicador	Resultado
Quantidade total de testes	[número]
Testes aprovados	[número]
Testes reprovados	[número]
Taxa de sucesso	[xx%]

4.2 Teste de integração

4.2.1 Integração com banco de dados

Demonstre que o sistema interage corretamente com o banco de dados, considerando as operações de criação, leitura, atualização e exclusão, quando aplicáveis.

ID	Integração testada	Operação	Resultado esperado	Resultado obtido
TI01	Aplicação ↔ Banco	Inserção	[Resultado]	[Resultado]
TI02	Aplicação ↔ Banco	Consulta	[Resultado]	[Resultado]
TI03	Aplicação ↔ Banco	Atualização	[Resultado]	[Resultado]
TI04	Aplicação ↔ Banco	Exclusão	[Resultado]	[Resultado]

4.2.2 Integração com APIs e serviços

Caso o projeto utilize APIs, serviços externos ou outros componentes, apresente os testes realizados. Caso não utilize, registre "Não se aplica ao projeto".

ID	Componente	Integração	Resultado esperado	Resultado obtido
TI05	[API/Serviço]	[Descrição]	[Resultado]	[Resultado]
TI06	[API/Serviço]	[Descrição]	[Resultado]	[Resultado]

4.3 Teste de usabilidade - Heurísticas de Nielsen

Avalie a interface desenvolvida considerando as 10 heurísticas de usabilidade de Jakob Nielsen.

Heurística	Atende?	Evidência/Justificativa
1. Visibilidade do status do sistema	Sim/Parcial/Não	[Descrição]
2. Correspondência entre sistema e mundo real	Sim/Parcial/Não	[Descrição]
3. Controle e liberdade do usuário	Sim/Parcial/Não	[Descrição]
4. Consistência e padrões	Sim/Parcial/Não	[Descrição]
5. Prevenção de erros	Sim/Parcial/Não	[Descrição]
6. Reconhecimento em vez de memorização	Sim/Parcial/Não	[Descrição]
7. Flexibilidade e eficiência de uso	Sim/Parcial/Não	[Descrição]
8. Design estético e minimalista	Sim/Parcial/Não	[Descrição]
9. Reconhecimento, diagnóstico e recuperação de erros	Sim/Parcial/Não	[Descrição]
10. Ajuda e documentação	Sim/Parcial/Não	[Descrição]

Problemas de usabilidade encontrados

ID	Problema identificado	Heurística relacionada	Correção realizada
PU01	[Problema]	[Heurística]	[Correção]
PU02	[Problema]	[Heurística]	[Correção]

5. SITUAÇÃO FINAL DO MVP

Apresente uma síntese do que foi efetivamente entregue e demonstre a relação entre o que foi previsto e o que está funcionando no sistema.

5.1 Funcionalidades previstas x implementadas

Funcionalidade	Prevista	Implementada	Funcionando	Observação
[Funcionalidade 1]	Sim	Sim/Não	Sim/Não	[Observação]
[Funcionalidade 2]	Sim	Sim/Não	Sim/Não	[Observação]
[Funcionalidade 3]	Sim	Sim/Não	Sim/Não	[Observação]
[Funcionalidade 4]	Sim	Sim/Não	Sim/Não	[Observação]
[Funcionalidade 5]	Sim	Sim/Não	Sim/Não	[Observação]

5.2 Persistência dos dados

Explique como os dados da aplicação são armazenados e demonstre que permanecem disponíveis após o encerramento e nova execução da aplicação.

[Descreva e apresente evidências.]

5.3 Limitações conhecidas

Informe funcionalidades incompletas, problemas conhecidos ou limitações da versão apresentada.

- [Limitação 1]
- [Limitação 2]
- [Limitação 3]

6. CONSIDERAÇÕES FINAIS

Apresente uma breve análise sobre o desenvolvimento do projeto, abordando os resultados alcançados, dificuldades encontradas, decisões técnicas relevantes, aprendizados, possíveis melhorias e funcionalidades futuras.

[Escreva aqui as considerações finais.]

ANEXOS

Quando necessário, podem ser incluídos:

- diagramas em maior resolução;
- protótipos;
- scripts;
- evidências de testes;
- relatórios de cobertura;
- imagens complementares;
- documentação de APIs;
- demais evidências técnicas.

ORIENTAÇÕES PARA ENTREGA

- O documento deverá representar o que foi efetivamente desenvolvido pelo grupo.
- Não deverão ser apresentadas como concluídas funcionalidades que não estejam

implementadas no software.

- Todas as funcionalidades descritas como implementadas deverão poder ser demonstradas durante a apresentação do projeto.
- Diagramas, requisitos, banco de dados, interfaces e código deverão possuir coerência entre si.
- O código-fonte deverá estar disponível no repositório indicado no documento.
- O MVP deverá ser executável e permitir a demonstração das principais funcionalidades propostas.
- A persistência dos dados deverá ser demonstrada quando fizer parte dos requisitos da solução.
- Falhas, limitações ou itens não concluídos devem ser informados de forma explícita na seção “Situação Final do MVP”.

A existência apenas de documentação, protótipos, telas isoladas, scripts ou trechos de código não caracteriza a conclusão do projeto. Os diferentes componentes da solução deverão estar integrados e permitir a execução do fluxo principal do sistema.